

Dinamikus Programozás

Mint problémamegoldási stratégia

Szerző: Nikházy László
Előadó: Gyimesi Péter

2026. június 8.

Horváth Gyula és Zsakó László diasorainak felhasználásával

Licit – bevétel maximalizálása

Egy földterületet M darab egységnyi parcellára osztottak. N pályázó adott be ajánlatot: mindegyik megad egy intervallumot

$$[A, B] \quad (1 \leq A \leq B \leq M)$$

és egy ajánlati árat F forintban. Minden parcella **legfeljebb egy** nyertes ajánlatban szerepelhet. Mekkora lehet az elérhető legnagyobb összbevétel, és melyik ajánlatokat kell elfogadni ehhez?

Korlátok: $1 \leq N$, $M \leq 1000$, az ajánlati árak $1 \leq F \leq 1000$.

Példa: $N = 4$, $M = 5$.

Ajánlatok (A, B, F) : $(2, 3, 500)$, $(4, 4, 600)$, $(4, 5, 500)$, $(1, 5, 1000)$.

A legjobb választás: az 1. és 2. ajánlat (összbevétel: $500 + 600 = 1100$).

Észrevétel: Ha az optimális megoldásból kihagyjuk az utolsó ajánlatot, akkor kezdete előtti parcellákra egy optimális megoldást alkot a többi ajánlat.

Részproblémák: $dp[i]$ a legnagyobb bevétel, ha csak az 1-től i -ig terjedő parcellákat használhatjuk fel.

Kezdeti érték: $dp[0] = 0$ (üres földszávr a bevétel is nulla)

Átmenet: Az ajánlatokat a B szerinti sorrendben dolgozzuk fel, egy aktuális (A, B, F) ajánlattal frissítjük a $dp[B]$ értéket:

$$dp[B] = \max(dp[B], dp[A - 1] + F)$$

(Azaz: ha elfogadjuk ezt az ajánlatot, akkor a bal oldali részre $dp[A - 1]$ a legjobb érték, és ehhez jön az ajánlati ár F .)

Válasz: $dp[M]$ – az összes parcella felhasználásával elérhető maximális bevétel.

Időkomplexitás: $O(N + M)$, ha a beolvasás után minden B végponthoz listázzuk az ajánlatokat.

Rekonstrukció ötlete:

- Tároljuk, hogy **melyik ajánlattal értük el** a $dp[i]$ értéket: egy $choice[i]$ tömbben (0, ha egyikkel sem).
- Visszafelé haladunk $i = M, M - 1, \dots, 1$:
 - ha $choice[i] = 0$, akkor $i \leftarrow i - 1$;
 - ha $choice[i] = k$ (azaz a k -adik ajánlatot választottuk), akkor felvesszük k -t a megoldásba, és $i \leftarrow A_k - 1$.

Licit – Kódvázlat

```
1  int N, M;
2  cin >> N >> M;
3  vector<int> A(N+1), B(N+1), F(N+1);
4  vector<vector<int>> endsAt(M+1);
5  for (int i = 1; i <= N; i++) {
6      cin >> A[i] >> B[i] >> F[i];
7      endsAt[B[i]].push_back(i);
8  }
9  vector<int> dp(M+1), choice(M+1);
10 for (int i = 1; i <= M; i++) {
11     dp[i] = dp[i-1];
12     for (int id : endsAt[i]) {
13         if (dp[A[id] - 1] + F[id] > dp[i]) {
14             dp[i] = dp[A[id] - 1] + F[id];
15             choice[i] = id;
16         }
17     }
18 }
19 // rekonstrukcio
20 vector<int> used;
21 int pos = M;
22 while (pos > 0) {
23     if (choice[pos] == 0) {
24         pos--;
25     } else {
26         int id = choice[pos];
27         used.push_back(id);
28         pos = A[id] - 1;
29     }
30 }
```

Gyöngyök csapdákkal – feladat

Egy $N \times M$ táblán minden mező vagy **gyöngyöket** tartalmaz, vagy **csapda** (-1). A bábu a bal felső sarokból indul, a jobb alsóba kell eljutni.

Lépések: csak **jobbra (J)** vagy **lefelé (L)** lehet menni, csapdára lépni nem szabad. A bejárt mezők gyöngyeit összegyűjtjük.

Feladat: a megszerezhető **maximális** gyöngyszám kiszámítása, és egy ehhez tartozó útvonal megadása (J/L lépések).

Korlátok: $1 \leq N, M \leq 1000$

Példa (optimális út jelölve):

1	2	3	4	0	1
2	-1	2	1	-1	3
-1	0	6	0	0	0
4	1	0	-1	1	-1
0	0	1	2	0	0

Maximális gyöngy: **17** Egy optimális út: J J L L L L J J J

Gyöngy csapdával – Megoldás

Észrevétel: Az optimális útvonal bármely köztes mezőjéig is optimális útvonalon kell eljutnunk.

Részproblémák: $dp[i][j]$ = maximálisan gyűjtött gyöngyszám az (i,j) mezőig. Ha az (i, j) mező csapda, vagy nem elérhető, akkor $dp[i][j]$ legyen „érvénytelen” ($-\infty$).

Átmenet: Csak **fentről** vagy **balról** jöhetünk:

$$dp[i][j] = \begin{cases} -\infty, & \text{ha } (i,j) \text{ csapda} \\ -\infty, & \text{ha } dp[i-1][j] = dp[i][j-1] = -\infty \\ \max(dp[i-1][j], dp[i][j-1]) + a[i][j], & \text{egyébként} \end{cases}$$

Ahol $a[i][j]$ a mezőn lévő gyöngyszám.

Kezdeti értékek:

$$dp[0][i] = dp[i][0] = -\infty, \quad dp[1][1] = a[1][1]$$

Válasz: $dp[N][M]$ (feltéve, hogy elérhető a jobb alsó sarok).

Idő- és tárigény: $O(N \cdot M)$ idő, $O(N \cdot M)$ memória.

Gyöngyök csapdákkal – DP táblázat

Példa bemenet:

1	2	3	4	0	1
2	-1	2	1	-1	3
-1	0	6	0	0	0
4	1	0	-1	1	-1
0	0	1	2	0	0

DP táblázat:

-	-	-	-	-	-	-
-	1	3	6	10	10	11
-	3	-	8	11	-	14
-	-	-	14	14	14	14
-	-	-	14	-	15	-
-	-	-	15	17	17	17

Jelölés:

- Az értékek a maximális összegyűjthető gyöngyöt mutatják az adott mezőig.
- - jelöli az elérhetetlen mezőt (csapda vagy blokkolt út).

A táblázat felrajzolásával világossá válik a kiszámítási sorrend is, jelenleg megfelel sorfolytonosan ($i = 1 \dots N$, $j = 1 \dots M$).

Gyöngy csapdával – Rekurzív megoldás?

Felmerülhet egy rekurzív megoldás, ahol a $G(i, j)$ függvény megadja, hogy legfeljebb hány gyöngy gyűjthető az (i, j) mezőig.

$$G(i, j) = \begin{cases} -\infty, & \text{ha } (i,j) \text{ csapda} \\ -\infty, & \text{ha } G(i-1, j) = G(i, j-1) = -\infty \\ \max(G(i-1, j), G(i, j-1) + a[i][j]), & \text{egyébként.} \end{cases}$$

Mi ezzel a probléma? A futási idő exponenciális, hiszen egy-egy mezőre nagyon sokszor meghívódik a függvény. Végeredményben annyi lépést csinál, ahány lehetséges útvonal van. (Csapda nélküli tábla esetében $\binom{N+M-2}{N-1}$.)

Gyöngy csapdával – Rekonstrukció

Cél: egy konkrét maximális útvonalat is ki kell írni (J/L karakterek sorozata).

Ötlet: A DP töltése közben elmentjük, hogy honnan jöttünk a (i, j) mezőre:

- egy `prev[i][j]` változó:
 - 'L' = fentről (az $(i - 1, j)$ mezőről) jöttünk lefele lépéssel,
 - 'J' = balról (az $(i, j - 1)$ mezőről) jöttünk jobbra lépéssel,
- csak akkor állítjuk be, ha `dp[i][j]` érvényes.

Visszafejtés:

- kiindulunk a (N, M) mezőről,
- amíg nem érünk $(1, 1)$ -be: az útvonalhoz hozzáadjuk a `prev[i][j]` által jelölt lépést, és annak megfelelően lépünk fel vagy balra.
- a lépéseket fordított sorrendben kapjuk, a végén megfordítjuk a karakterláncot.

Pénzváltás minimális érmével

Egy pénzrendszerben N darab különböző címletű érme van ($1 \leq N \leq 100$). Minden érme pozitív egész értékű:

$C_1, C_2, \dots, C_N$ ($1 \leq C_i \leq 10^6$).

Egy X összeg ($1 \leq X \leq 10^6$) kifizetését kell megadni úgy, hogy:

- az érmek száma **minimális** legyen,
- és állítsuk is elő egy konkrét **felbontással** az összeget.

Ha az összeg nem állítható elő, a kimenet: -1.

Példa:

$N = 3, X = 11$ Címletek: $(C_1, C_2, C_3) = (1, 5, 7)$

Optimális felbontás: $11 = 5 + 5 + 1$ (3 érme).

Kimenet: 3

5 5 1

Pénzváltás minimális értékel – Megoldás

Észrevétel: Ha az optimális megoldásból bármelyik értékét elhagyjuk, akkor az annival csökkentett összegre is optimális felváltást kapunk.

Részproblémák: $dp[s]$ = minimális érték szám az s összegre.

Rekonstrukcióhoz segéd tömb: $prev[s]$ = melyik érték vezetett ide.

Alapeset: $dp[0] = 0$, $prev[0] = -1$.

Minden más s -re kezdetben $dp[s] = \infty$.

Átmenet: $s = 1 \dots X$, $i = 1 \dots N$:

ha $s \geq C_i$ és $dp[s - C_i] + 1 < dp[s]$, akkor:

$dp[s] = dp[s - C_i] + 1$, $prev[s] = C_i$.

Válasz:

- ha $dp[X] = \infty$, akkor -1 ,
- különben $dp[X]$ és a **rekonstruált felbontás**.

Komplexitás:

$O(N \cdot X)$ idő, $O(X)$ memória.

Pénzváltás minimális érmével – Rekonstrukció

A `prev[]` tömb megmondja, hogy `dp[s]` értékét melyik érme adta:

- indulunk $s = X$ -től,
 - amíg $s > 0$:
 - a felhasznált érme: `prev[s]`,
 - levonjuk: $s \leftarrow s - \text{prev}[s]$,
 - a kapott érméket megfordítjuk (fordított sorrendben gyűjtöttük).
- Ez pontosan előállítja az egyik minimális érmékből álló felbontást.

Pénzváltás minimális érmével – Kódvázlat

```
1  int N, X;
2  cin >> N >> X;
3  vector<int> C(N);
4  for (int i = 0; i < N; i++) cin >> C[i];
5  const int INF = 1e9;
6  vector<int> dp(X + 1, INF), prev(X + 1, -1);
7  dp[0] = 0;
8
9  for (int coin : C) {
10     for (int s = coin; s <= X; s++) {
11         if (dp[s - coin] + 1 < dp[s]) {
12             dp[s] = dp[s - coin] + 1;
13             prev[s] = coin;
14         }
15     }
16 }
17
18 if (dp[X] == INF) {
19     cout << -1 << "\n";
20 } else {
21     cout << dp[X] << "\n";
22
23     vector<int> coins;
24     int s = X;
25     while (s > 0) {
26         coins.push_back(prev[s]);
27         s -= prev[s];
28     }
29     reverse(coins.begin(), coins.end());
30
31     for (int x : coins) cout << x << "_";
32     cout << "\n";
33 }
```

Adott N darab tárgy ($1 \leq N \leq 2000$). Minden tárgynak ismert a súlya (w_i) és az értéke (v_i). Van egy hátizsákunk, amely legfeljebb C összsúlyt bír el ($1 \leq C \leq 2000$). Legfeljebb mennyi összértékű tárgyat tudunk kiválasztani úgy, hogy az összsúlyuk ne haladja meg a C kapacitást?

Példa:

$N = 4, C = 10$.

Tárgyak: $(w, v) = (3, 4), (5, 8), (5, 7), (8, 14)$.

Optimális választás: az 2. és 3. tárgy együttesen (súly $5 + 5 = 10$, érték $8 + 7 = 15$).

Hátizsák-probléma – Megoldás

Észrevétel: Ha az optimális megoldásból kihagyunk egy tárgyat, akkor a maradék a tárgy súlyával csökkentett méretű hátizsákra optimális megoldás *a többi tárgyból választva*.

Részproblémák: $dp[i][c]$ = maximális érték, ha csak az első i tárgy közül választhatunk, és a hátizsák kapacitása c .

Alapesetek:

- $dp[0][c] = 0$ minden $0 \leq c \leq C$ (nincs tárgy $\Rightarrow$ 0 érték).
- $dp[i][0] = 0$ minden $0 \leq i \leq N$ (0 kapacitás $\Rightarrow$ 0 érték).

Átmenet: az i -edik tárgyat vagy nem vesszük be, vagy bevesszük (ha belefér).

$$dp[i][c] = \begin{cases} dp[i-1][c], & \text{ha } w_i > c \\ \max(dp[i-1][c], dp[i-1][c - w_i] + v_i), & \text{ha } w_i \leq c \end{cases}$$

Válasz: $dp[N][C]$.

Idő- és tárigény: $O(N \cdot C)$ idő, $O(N \cdot C)$ memória.

Hátizsák-probléma – Optimalizálás és kódvázlat

Memóriaoptimalizálás (1D DP): Az $dp[i][\cdot]$ sor csak az $dp[i-1][\cdot]$ értékektől függ, ezért elég egy dimenzió:

$dp[c] = \max$ összérték, ha a kapacitás c .

Bejárás sorrendje: Itt **visszafelé** kell menni a kapacitáson, hogy ugyanazt a tárgyat ne használjuk többször:

$i = 1..N, c = C, C-1, \dots, w_i : dp[c] = \max(dp[c], dp[c-w_i] + v_i)$

C++ kódvázlat:

```
1  int N, C;
2  vector<int> w(N+1), v(N+1);
3  vector<long long> dp(C+1, 0);
4
5  for (int i = 1; i <= N; i++) {
6      for (int c = C; c >= w[i]; c--) {
7          dp[c] = max(dp[c], dp[c - w[i]] + v[i]);
8      }
9  }
10 // Valasz: dp[C]
```

Rekonstrukció (mely tárgyakat választottuk?)

A 2D táblát kell használnunk és visszalépegetünk $i = N \dots 1$, c -t követve:

ha $dp[i][c] = dp[i-1][c] \Rightarrow i$ nincs bent,

különben i bent van, és $c \leftarrow c - w_i$.

A dinamikus programozás stratégiája

A dinamikus programozás, mint probléma-megoldási stratégia az alábbi öt lépés végrehajtását jelenti.

1. Az [optimális] megoldás szerkezetének elemzése.

2. Részproblémákra és összetevőkre bontás úgy, hogy:

- a) az összetevőktől való függés körmentes legyen,
- b) minden részprobléma [optimális] megoldása kifejezhető legyen (rekurzívan) az összetevők [optimális] megoldásaival.

3. Részproblémák [optimális] megoldásának rekurzív kifejezése az összetevők [optimális] megoldásaiból.

4. Részproblémák [optimális] megoldásának kiszámítása alulról-felfelé:

- a) a kiszámítási sorrend meghatározása (minden részprobléma összetevői előbb szerepeljenek),
- b) a részproblémák kiszámítása táblázat-kitöltéssel.

5. Egy [optimális] megoldás előállítása 4. lépésben kiszámított (és tárolt) információkból.

Rúd darabolása – Feladat

Adott egy H hosszúságú fémrúd. A rudat N megadott helyen kell elvágni ($N \leq 1000$). A vágási pontok: $v_1 < v_2 < \dots < v_N$.

A vágásokat bármilyen sorrendben elvégezhetjük. Egy vágás költsége mindig megegyezik **annak a darabnak a hosszával**, amelyet éppen két részre vágunk.

Feladat: Számítsuk ki a minimális összköltséget, és adjunk meg egy olyan vágási sorrendet, amely ezt eléri.

Példa:

Bemenet:

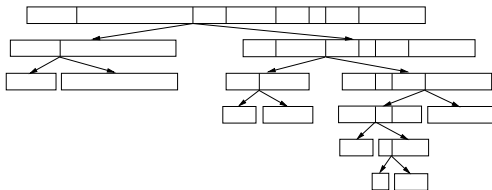
24 7

3 10 12 15 17 18 20

Kimenet:

70

2 1 4 3 7 5 6



Rúd darabolása – Megoldás

Szerkezet: Egészítsük ki a vágási pontokat: $v_0 = 0$, $v_{N+1} = H$.

Ha az optimális vágási sorozatban az első vágás a v_k pont, akkor a $[v_0, v_k]$ és $[v_k, v_{N+1}]$ részeket is optimálisan vágjuk fel.

Részproblémák:

$dp[i][j] = a[v_i, v_j]$ intervallum teljes optimális vágási költsége ($0 \leq i < j \leq N + 1$). Ha nincs benne vágási pont ($j = i + 1$), akkor $dp[i][j] = 0$.

Rúd darabolása – Megoldás

Szerkezet: Egészítsük ki a vágási pontokat: $v_0 = 0$, $v_{N+1} = H$.

Ha az optimális vágási sorozatban az első vágás a v_k pont, akkor a $[v_0, v_k]$ és $[v_k, v_{N+1}]$ részeket is optimálisan vágjuk fel.

Részproblémák:

$dp[i][j] = a[v_i, v_j]$ intervallum teljes optimális vágási költsége ($0 \leq i < j \leq N+1$). Ha nincs benne vágási pont ($j = i+1$), akkor $dp[i][j] = 0$.

Átmenet: Az első vágás helye legyen k (ahol $i < k < j$). Ekkor:

$$dp[i][j] = (v_j - v_i) + \min_{k=i+1}^{j-1} (dp[i][k] + dp[k][j]).$$

A minimumot elérő k -t tároljuk:

$$firstcut[i][j] = \arg \min_{k=i+1}^{j-1} (dp[i][k] + dp[k][j]).$$

Rekonstrukció:

- induljunk a teljes intervallumból: $[0, N + 1]$,
- az első vágási hely: $k = \text{firstcut}[0][N + 1]$,
- majd rekurzívan feldolgozzuk a $[0, k]$ és a $[k, N + 1]$ intervallumokat,
- az így kapott k -k sorozata adja a vágások optimális sorrendjét.

Projects – Feladat (CSES)

Adott n projekt. A i -edik projekt adatai: kezdő nap a_i , záró nap b_i , jutalom p_i . Egy napon **legfeljebb egy** projekten vehetünk részt.

Feladat: válasszunk ki néhány, egymással **nem átfedő** projektet úgy, hogy az összjutalom **maximális** legyen. Kimenet: a maximális összeg.

Korlátok: $1 \leq n \leq 2 \cdot 10^5$, $1 \leq a_i \leq b_i \leq 10^9$, $1 \leq p_i \leq 10^9$.

Példa: Bemenet:

```
4
2 4 4
3 6 6
6 8 2
5 7 3
```

Egy optimális választás:

1. projekt (2–4, 4 egység) és

4. projekt (5–7, 3 egység)

→ összjutalom: **7**.

Kimenet:

7

Kapcsolat a Licit feladattal: Ugyanaz a probléma: nem átfedő intervallumok közül kell maximum összértékű halmazt választani.

Különbség: itt a napok akár 10^9 -ig terjednek, ezért nem lehet $dp[nap]$ tömböt használni.

Kapcsolat a Licit feladattal: Ugyanaz a probléma: nem átfedő intervallumok közül kell maximum összértékű halmazt választani.

Különbség: itt a napok akár 10^9 -ig terjednek, ezért nem lehet $dp[nap]$ tömböt használni.

Szerkezet: Rendezzük a projekteket a befejezési nap szerint (b_i). Az optimális megoldás utolsó projektje legyen i ; előtte csak olyan projekt lehet, amely a_i előtt befejeződik.

Részproblémák:

$dp[i] =$ a maximális összjutalom az első i (rendezett) projektből.

Átmenet:

Kapcsolat a Licit feladattal: Ugyanaz a probléma: nem átfedő intervallumok közül kell maximum összértékű halmazt választani.

Különbség: itt a napok akár 10^9 -ig terjednek, ezért nem lehet $dp[nap]$ tömböt használni.

Szerkezet: Rendezzük a projekteket a befejezési nap szerint (b_i). Az optimális megoldás utolsó projektje legyen i ; előtte csak olyan projekt lehet, amely a_i előtt befejeződik.

Részproblémák:

$dp[i]$ = a maximális összjutalom az első i (rendezett) projektből.

Átmenet:

Legyen $prev[i]$ = utolsó projekt, amelyre $b_{prev[i]} < a_i$. Ekkor:

$$dp[i] = \max(dp[i-1], dp[prev[i]] + p_i).$$

Projects – Számítási sorrend és rekonstrukció

Számítási sorrend:

- rendezzük a projekteket befejezési nap szerint (b_i),
- a rendezési utáni $i = 1 \dots n$ sorrendben töltjük ki a $dp[i]$ értékeket.
- **DP-lépés:** $dp[i] = \max(dp[i-1], dp[prev[i]] + p_i)$.

prev[i] meghatározása:

Projects – Számítási sorrend és rekonstrukció

Számítási sorrend:

- rendezzük a projekteket befejezési nap szerint (b_i),
- a rendezési utáni $i = 1 \dots n$ sorrendben töltjük ki a $dp[i]$ értékeket.
- **DP-lépés:** $dp[i] = \max(dp[i-1], dp[prev[i]] + p_i)$.

$prev[i]$ meghatározása:

- $prev[i] =$ legnagyobb index, amire $b_{prev[i]} < a_i$,
- meghatározható:
 - **bináris kereséssel** a rendezett b -listában, vagy
 - **two pointers** módszerrel (lineáris végigfutás).

Rekonstrukció (ha kellene):

Projects – Számítási sorrend és rekonstrukció

Számítási sorrend:

- rendezzük a projekteket befejezési nap szerint (b_i),
- a rendezési utáni $i = 1 \dots n$ sorrendben töltjük ki a $dp[i]$ értékeket.
- **DP-lépés:** $dp[i] = \max(dp[i-1], dp[prev[i]] + p_i)$.

$prev[i]$ meghatározása:

- $prev[i] =$ legnagyobb index, amire $b_{prev[i]} < a_i$,
- meghatározható:
 - **bináris kereséssel** a rendezett b -listában, vagy
 - **two pointers** módszerrel (lineáris végigfutás).

Rekonstrukció (ha kellene): Hátrafelé indulva:

- ha $dp[i] = dp[i-1] \rightarrow$ projekt i kimarad,
- ha $dp[i] = dp[prev[i]] + p_i \rightarrow$ projekt i benne van, majd $i \leftarrow prev[i]$,

Komplexitás: rendezés $O(n \log n)$, $prev$ -ek meghatározása $O(n)$ vagy $O(n \log n)$, DP $O(n)$, memória $O(n)$.

Hegymászó – Feladat

Egy hegycsúcs megmászásához N tábor van az útvonalon ($1 \leq N \leq 1000$). A csúcs távolsága a kezdőponttól: T ($1 \leq T \leq 100000$). A hegymászó hátizsákja H kapacitású ($1 \leq H \leq 5000$), vagyis egyszerre legfeljebb H km-re elég élelmiszert vihet magával.

A táborok távolságai a kezdőponttól (növekvő sorrendben):

$D_1, D_2, \dots, D_N$ ($0 = D_1 < D_i < T$), és az ott elérhető élelem mennyisége: $F_1, F_2, \dots, F_N$ ($1 \leq F_i \leq 5000$). Az első tábor a kezdőpont: $D_1 = 0$. A bemenet biztosítja, hogy a csúcs elérhető.

Kérdés: Minimum hány táborban kell megállnia feltölteni a hátizsákját, és mik ezeknek a táboroknak a sorszámai?

Példa: $N = 4$, $T = 600$, $H = 300$

Táborok:

$(D, F) = (0, 200), (100, 300), (300, 300), (400, 200)$

Egy optimális megoldás: **3** megállás az **1.**, **2.** és **4.** táborban.

Hegymászó – Szerkezet és részproblémák

Szerkezet elemzése:

A hegymászó minden táborban kétféle döntést hozhat:

- **nem áll meg**, csak továbbmegy, vagy
- **megáll**, és *maximálisan* feltölti a hátizsákot (az elérhető ételmennyiséggel, legfeljebb H -ig). Nem érdemes részben feltölteni.
- Az optimális döntés attól függ, milyen tartalékkal érkeztünk.
- Az aktuális helyzetet három mennyiség jellemzi: (1) hányadik tábornál vagyunk (2) hányszor álltunk meg eddig, és (3) mennyi élelem van a hátizsákunkban.

Részproblémák: $dp[i][k]$ = a maximális ételmennyiség, amellyel elhagyhatjuk az i -edik tábornál, ha eddig pontosan k alkalommal álltunk meg élelmet felvenni.

Ha egy állapot nem érhető el: $dp[i][k] = -\infty$.

Kezdeti értékek: Az első tábor a kezdőpont ($D_1 = 0$), ezért

$$dp[1][1] = \min(H, F_1), \quad dp[1][k > 1] = -\infty.$$

Hegymászó – Megoldás

Jelölés: a két szomszédos tábor közti távolság $\Delta_i = D_i - D_{i-1}$.

A $dp[i][k]$ értéke **két lehetőség** közül kerül ki:

1. Nem állunk meg az i -edik táborban.

Akkor lehetséges, ha el tudunk jutni ide: $dp[i-1][k] \geq \Delta_i$.

Ekkor a maradék élelem:

$$option_1 = dp[i-1][k] - \Delta_i$$

2. Megállunk az i -edik táborban (ez a k -edik megállás).

Akkor lehetséges, ha $dp[i-1][k-1] \geq \Delta_i$.

Érkezés után marad: $dp[i-1][k-1] - \Delta_i$, majd ebből feltöltünk F_i -t, de legfeljebb a kapacitásig:

$$option_2 = \min(H, dp[i-1][k-1] - \Delta_i + F_i).$$

A kettő közül a jobbat választjuk:

$$dp[i][k] = \max(option_1, option_2),$$

csak a *érvényes* jelöltek közül (amelyekre előfeltétel teljesül).

Számítási sorrend:

- kényelmes, ha bevezetünk egy **mesterséges** $(N + 1)$ -edik tábort: $D_{N+1} = T$, $F_{N+1} = 0$,
- $i = 2 \dots N + 1$, azon belül $k = 1 \dots i$ -re számítjuk a $dp[i][k]$ -t az előző dián leírt átmenettel.

Válasz:

- a csúcs elérhető, ha $dp[N + 1][k]$ érvényes (nem $-\infty$),
- keressük a **legkisebb** K -t, amelyre $dp[N + 1][K]$ érvényes, ez a minimális megállások száma.

Rekonstrukció:

- a `dp` mellett egy `stop[i][k]` jelölésben eltároljuk, hogy `dp[i][k]`-t a „megállunk az i -edik táborban” esetből vettük-e,
- kiindulunk az $(i, k) = (N+1, K)$ állapotból, és i -t csökkentve visszafelé haladunk:
 - ha `stop[i][k]` igaz és $i \leq N$, akkor a i -edik tábor bekerül a megállások közé, és $k \leftarrow k - 1$,
 - különben csak továbblépünk: $i \leftarrow i - 1$.
- a végén a megállások sorszámait megfordítjuk $\rightarrow$ növekvő sorrendben kapjuk.

Komplexitás: a `dp` (és `stop`) tábla mérete $O(N^2)$, a kitöltés $O(N^2)$ idő, memóriaigény $O(N^2)$.

Kitalálós játék nem-válasz korláttal – Feladat

Éva gondol egy számot 1 és N között. Ádám ilyen kérdéseket tehet fel:

„A gondolt szám kisebb-e, mint x ?”

A válasz mindig **igen** vagy **nem**.

Megkötés: Ádám legfeljebb H kérdésre kaphat „**nem**” választ, utána nem kérdezhet tovább.

Feladat: Adott N és H mellett keressük a legkisebb k -t, amelyre Ádám **bármely** $1 \dots N$ közötti számot ki tud találni legfeljebb k kérdéssel, és legfeljebb H darab „nem” válasz felhasználásával.

Korlátok: $1 \leq N \leq 2 \cdot 10^9$, $2 \leq H \leq 100$

Példa:

Bemenet: 9 2

Kimenet: 4

Kitalálós játék – Szerkezet (kérdőfa)

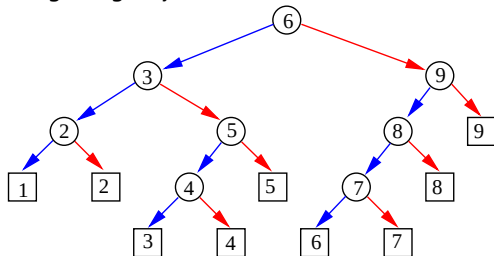
A stratégia leírható egy **bináris kérdőfával**:

- minden levél egy lehetséges gondolt számot jelöl ($1 \dots n$),
- minden belső csúcs egy kérdéshez tartozik (valamilyen x),
- bal él: válasz = „igen” ($< x$), jobb él: válasz = „nem” ($\geq x$).

Megkötések a fára:

- a fának legalább N levele van (minden számhoz külön levél),
- bármely gyökértől levélig vezető úton $\leq H$ -szor mehetünk jobbra,
- a fa magassága = legrosszabb esetben szükséges kérdések száma.

Cél: minimális magasságú ilyen kérdőfa keresése $\Rightarrow$ ez a keresett k .



Egy 2-hibázó kérdőfa a példa bemenetre.

Legyen $dp[k][h]$ annak a **legtöbb levélnek a száma**, amely elhelyezhető egy olyan kérdezőfában, amely legfeljebb k magas (legfeljebb k kérdés) és egy úton legfeljebb h darab „nem” válasz szerepel.

Értelmezés:

- k = engedélyezett maximális kérdésszám,
- h = engedélyezett maximális „nem” válasz egy úton.

Kapcsolat a feladattal: Az Ádám által biztosan kitalálható számok száma éppen $dp[k][H]$. A cél: a **legkisebb** k , amelyre $dp[k][H] \geq N$ teljesül.

Kitalálós játék – Megoldás

Alapeset: $dp[k][1] = k + 1$ (ha csak 1 „nem” lehet, akkor legfeljebb $k+1$ számot tudunk megkülönböztetni).

Ha a hibakeret nagyobb, mint a megengedett magasság:

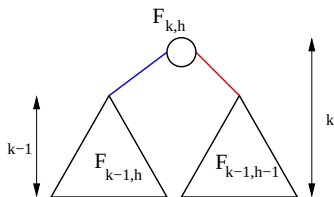
ha $k < h$, akkor $dp[k][h] = dp[k][k]$,

Általános rekurzió: ha $k > 1$ és $h \leq k$, akkor

$$dp[k][h] = dp[k-1][h] + dp[k-1][h-1].$$

Magyarázat:

- „igen” ág $\rightarrow$ ugyanannyi „nem”: $dp[k-1][h]$ levelet fed le,
- „nem” ág $\rightarrow$ eggyel kevesebb „nem”: $dp[k-1][h-1]$ levelet fed le.



$F_{k,h}$: k -magas h -hibázó legtöbb levelet tartalmazó kérdezőfa.

Kitalálós játék – Számítási sorrend és komplexitás

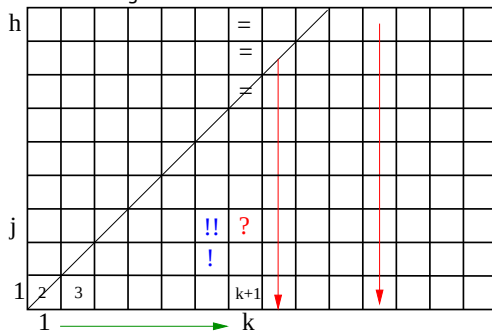
Számítási sorrend: A k magasságot növeljük: $k = 1, 2, 3, \dots$, és minden k -ra kiszámítjuk a $dp[k][h]$ értékeket, $h = 1 \dots \min(k, H)$.

Gyakorlatban egy dimenzió is elég: Egyetlen tömböt tartunk fenn:
 $dp[h] = dp[k][h]$. Új k esetén hátulról frissítjük:

$$dp[h] \leftarrow dp[h] + dp[h-1] \quad (h = \min(k, H), \dots, 2), \quad dp[1] = k+1.$$

Megállás: amint $dp[H] \geq N$, a jelenlegi k a megoldás.

Komplexitás: idő: $O(k_{\text{megoldás}} \cdot H)$, memória: $O(H)$.

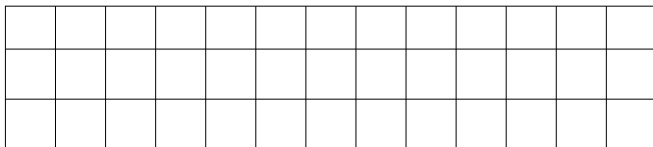


Részproblémák számítási sorrendje: oszloponként felülről lefelé haladva.

Kitalálós játék – Kódvázlat

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  long long dp[105];    // dp[h] = dp[k][h]
5
6  int main() {
7      long long N;
8      int H, k = 0;
9      cin >> N >> H;
10     while (dp[H] < N) {
11         k++;
12         // dp[k][h] = dp[k-1][h] + dp[k-1][h-1], hátulról frissítve
13         for (int h = min(H, k); h >= 2; h--) {
14             dp[h] = dp[h] + dp[h-1];
15         }
16         dp[1] = k + 1;    // dp[k][1] = k + 1
17         // h > k eset: másoljuk az utolsót
18         for (int h = k + 1; h <= H; h++) {
19             dp[h] = dp[h - 1];
20         }
21     }
22     cout << k << "\n";
23 }
```

Számítsuk ki, hogy hányféleképpen lehet egy $3 \times N$ egység méretű járdát 1×2 méretű lapokkal (dominókkal) teljesen lefedni.



Járdakövezés - Megoldás

A megoldás szerkezete: jelölje $dp[i]$, hogy hányféleképpen fedhető le egy $3 \times i$ egység méretű járda. Ki tudjuk fejezni $dp[i]$ -t a $dp[i-1]$, $dp[i-2]$, ... értékekből?

Járdakövezés - Megoldás

A megoldás szerkezete: jelölje $dp[i]$, hogy hányféleképpen fedhető le egy $3 \times i$ egység méretű járda. Ki tudjuk fejezni $dp[i]$ -t a $dp[i-1]$, $dp[i-2]$, ... értékekből?

Ehhez az kellene, hogy legyen egy függőleges választóvonal, amin nem megy át dominó, viszont megeshet, hogy nincs is ilyen vonal (lásd a jobb oldali példát). A megoldáshoz további részproblémát kell felvennünk.



Járdakövezés - Megoldás

A megoldás szerkezete: jelölje $dp[i]$, hogy hányféleképpen fedhető le egy $3 \times i$ egység méretű járda. Ki tudjuk fejezni $dp[i]$ -t a $dp[i-1]$, $dp[i-2]$, ... értékekből?

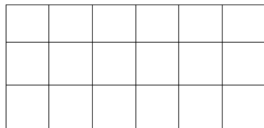
Ehhez az kellene, hogy legyen egy függőleges választóvonal, amin nem megy át dominó, viszont megeshet, hogy nincs is ilyen vonal (lásd a jobb oldali példát). A megoldáshoz további részproblémát kell felvennünk.



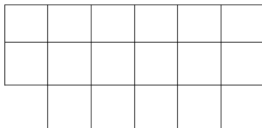
Részproblémák:

- $dpA[i]$ = hányféleképpen fedhető le egy $3 \times i$ méretű járda.
- $dpB[i]$ = hányféleképpen fedhető le egy $3 \times i$ méretű *hiányos* járda, amelynek a bal alsó sarokmezője hiányzik.

Így már elvégezhető a visszavezetés kisebb részproblémákra.

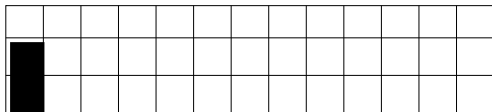
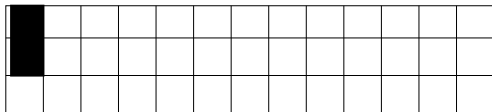
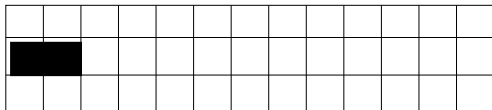


A típusú (teljes) járda



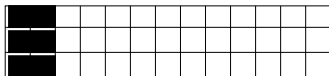
B típusú (hiányos) járda

Vizsgáljuk, hogy az első oszlop középső mezőjét milyen dominó fedi le. Három féle lehet:

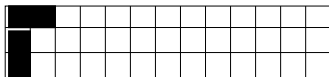
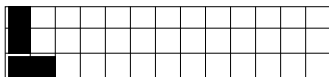


Járdakövezés - Átmenet, teljes járda

Az első esetet csak az alábbi módon lehet folytatni. A fennmaradó részt $dpA[i - 2]$ féleképpen lehet lefedni, mivel egy $i - 2$ hosszúságú teljes járda.



A második és harmadik eset is csak egyféleképpen folytatható, és egy $i - 1$ hosszúságú hiányos járda marad, amit $dpB[i - 1]$ féleképpen lehet lefedni.



Rekurzív összefüggés:

$$dpA[i] = dpA[i - 2] + 2 \cdot dpB[i - 1], \quad i = 2 \dots N$$

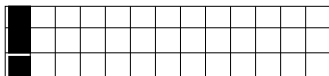
Alapesetek:

$dpA[0] = 1$, hiszen a semmit egyféleképpen lehet lefedni.

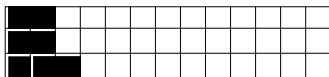
$dpA[1] = 0$, mert az 1×3 járda nem fedhető le.

Járdakövezés - Átmenet, hiányos járda

Hiányos járdánál csak két eset van, az első esetben egy $i - 1$ méretű teljes járda marad:



A másik eset egy $i - 2$ méretű hiányos járdára vezethető vissza:



Rekurzív összefüggés:

$$dpB[i] = dpA[i - 1] + dpB[i - 2], \quad i = 2 \dots N$$

Alapesetek:

$dpB[0] = 0$, hiszen nem is létezik 0 hosszúságú hiányos járda (a semmiből nem hiányozhat egy mező).

$dpB[1] = 1$, mert az 1 hosszúságú hiányos járda pont lefedhető egy függőleges dominóval.

Járdakövezés - Kód

A megoldás során kétutas dinamikus programozást használtunk, ahol az A és B típusú DP értékek egymásból számolódnak.

Komplexitás: $O(N)$ idő- és tárigény.

Kód:

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  int main() {
4      int N; cin >> N;
5      vector<long long> A(N+1, 0), B(N+1, 0);
6      A[0] = 1; A[1] = 0;
7      B[0] = 0; B[1] = 1;
8      for (int i = 2; i <= N; ++i) {
9          B[i] = A[i-1] + B[i-2];
10         A[i] = A[i-2] + 2 * B[i-1];
11     }
12     cout << A[N] << "\n";
13     return 0;
14 }
```

Igazságos osztzkodás – Feladat

Két testvér közösen kapott N ajándékot ($N \leq 300$). Minden ajándéknak ismert a használati értéke, pozitív egész szám. Az ajándékok összértéke legfeljebb 10000.

Cél: igazságosan elosztani az ajándékokat úgy, hogy mindketten **azonos összértékű** csomagot kapjanak, viszont maradhat ajándék a közösben.

Mennyi lehet a **közösben maradó** ajándékok **legkisebb** összértéke, és hogy kell szétosztaniuk az ajándékokat?

Példa bemenet:

```
6
10 3 12 5 15 6
```

Kimenet:

```
15
3 6
1 2 4
```

Magyarázat:

A közösben maradó ajándékok összértéke: 15. Az egyik testvér a 3. és 6. ajándékot kapja, $12 + 6 = 18$ összértékben, a másik az 1., 2. és 4. ajándékot $10 + 3 + 5 = 18$ összértékben.

Részproblémák:

$dp[i][d]$ = a **minimális közösben maradó összeg**, amelyet elérhetünk az első i ajándék feldolgozásával, ha d a két testvér által kapott összértékek különbsége: $d = \text{sum}_A - \text{sum}_B$.

A megoldás: $dp[N][0]$

Trükk: negatív indexek. A fenti definíció megenged d -re negatív értékeket. Hogy ez ne jelentsen gondot, az indexelést eltoljuk az összes ajándék összértékével.

Minden ajándéknál három döntési lehetőség:

- A kapja az i -edik ajándékot $\Rightarrow d$ nő v_i -vel,
- B kapja $\Rightarrow d$ csökken v_i -vel,
- közösbe kerül $\Rightarrow d$ változatlan, de a közösösszeg nő.

Igazságos osztzkodás – Rekurzív összefüggések

Jelölje v_i az i -edik ajándék értékét. A d különbség tömbindexe:
 $idx = d + S$ (ahol S az összeg).

Átmenetek:

$$dp[i][d] = \min \begin{cases} dp[i-1][d] + v_i, & \text{(közösbe tesszük)} \\ dp[i-1][d - v_i], & \text{(A kapja)} \\ dp[i-1][d + v_i] & \text{(B kapja)} \end{cases}$$

Alapeset:

$$dp[0][0] = 0, \quad dp[0][d \neq 0] = \infty.$$

Kiszámítási sorrend:

- először $dp[0][\cdot]$ sor, majd $dp[1][\cdot]$, $dp[2][\cdot]$, $\dots$, $dp[N][\cdot]$,
- minden sorban a d értékeket teljes tartományban bejárjuk ($-S \dots S$).

Komplexitás: $O(N \cdot S)$

A felosztás előállítható a DP-tábla alapján.

Visszafelé haladunk: az $i = N, N - 1, \dots, 1$ sorokban.

Ha jelenlegi különbség d és érték $dp[i][d]$:

- ha $dp[i][d] = dp[i - 1][d] + v_i$, akkor a tárgy **közösben maradt**;
- ha $dp[i][d] = dp[i - 1][d - v_i]$, akkor az i -edik tárgyat **A kapta**, új $d \leftarrow d - v_i$;
- ha $dp[i][d] = dp[i - 1][d + v_i]$, akkor az i -edik tárgyat **B kapta**, új $d \leftarrow d + v_i$.

Feladat: Adott egy N pozitív egész szám. N egy *partíciója* egy olyan $a_1 \geq a_2 \geq \dots \geq a_k > 0$ sorozat, amely elemeinek összege $a_1 + a_2 + \dots + a_k = N$.

Kérdés:

$P(N) =$ hány különböző partíciója van az N számnak?

Példa:

- $P(4) = 5$, a partíciók: $4, 3 + 1, 2 + 2, 2 + 1 + 1, 1 + 1 + 1 + 1$

Részproblémákra bontás:

$dp[i][k]$ = a partíciók száma az i -re úgy, hogy minden tag legfeljebb k .

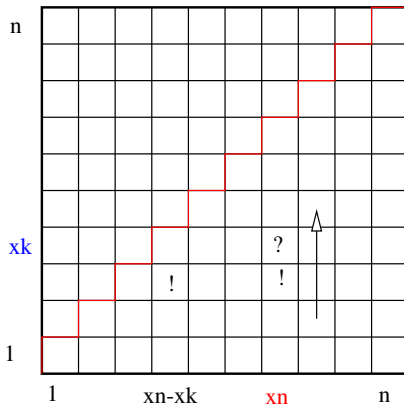
Alapesetek: $dp[0][k] = 1 \quad \forall k \geq 0$

Összefüggés:

- Ha $i < k$, akkor $dp[i][k] = dp[i][i]$
- Különben: $dp[i][k] = dp[i][k-1] + dp[i-k][k]$

Partíciószám – DP számítási sorrend

$$dp[i][k] = dp[i][k-1] + dp[i-k][k]$$



Komplexitás: $O(N^2)$ idő és $O(N^2)$ memória.

Partíciós szám – Kód

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main() {
5      int N; cin >> N;
6      vector<vector<long long>> dp(N+1, vector<long long>(N+1, 0)
7      for (int k = 0; k <= N; k++) dp[0][k] = 1;
8      for (int i = 1; i <= N; i++) {
9          for (int k = 1; k <= N; k++) {
10             if (k > i) {
11                 dp[i][k] = dp[i][i];
12             } else {
13                 dp[i][k] = dp[i][k-1] + dp[i - k][k];
14             }
15         }
16     }
17     cout << dp[N][N] << "\n";
18 }
```

Feladatok

Önálló munkára

Fazekas kemence súlykorláttal – Feladat

<https://mester.inf.elte.hu> → Dinamikus programozás / 20.

Sorban érkeznek a kiégetendő tárgyak. A i -edik tárgy égetési ideje E_i , súlya S_i . A tárgyakat érkezési sorrendben kell feldolgozni. Egy menetben egyszerre több tárgy is égethető, de összsúlyuk nem haladhatja meg a kemence kapacitását (K). Egy menet égetési ideje: a benne lévő tárgyak maximális égetési ideje. Mennyi a minimális összégetési idő, és ehhez mely tárgyakat kell együtt égetni?

Bemenet: N (tárgyak száma, $1 \leq N \leq 10000$), K (kemence kapacitása, $1 \leq K \leq 100$). Majd N sor: E_i (égetési idő, $1 \leq E_i \leq 200$), S_i (súly, $1 \leq S_i \leq K$).

Kimenet: A minimális összégetési idő, majd soronként egy-egy menet: az első és utolsó tárgy sorszáma.

Példa bemenet:

```
7 3
10 1
8 2
20 1
25 1
30 1
12 1
40 1
```

Példa kimenet:

```
75
1 2
3 4
5 7
```

Magyarázat: az első két tárgy égetése 10 percig tart, a harmadik és negyedik 25 perc alatt készül el, majd az utolsó három még 40 perc alatt.

<https://mester.inf.elte.hu> → Dinamikus programozás / 77.

Egy autópálya mentén N város helyezkedik el. Bizonyos városokban rendőrök tartózkodnak, némelyikben több is, némelyikben egy sem. Összesen legfeljebb N rendőr van. Cél: a lehető legtöbb városban legyen rendőr, és az átcsoportosítást a lehető legkisebb összköltséggel kell végrehajtani. Egy rendőr i -ből j -be történő mozgatásának költsége $|i - j|$.

Bemenet: Első sor: N ($1 \leq N \leq 500$). Második sor: N szám, az i -edik városban kezdetben hány rendőr van (összesen $\leq N$)

Kimenet: Első sor: az átcsoportosítás legkisebb összköltsége. Második sor: N szám, 1 ha lesz rendőr, 0 egyébként.

Példa:

Bemenet

7
0 1 0 3 2 0 0

Kimenet

5
1 1 1 1 1 0 0

CEOI 2009 Photo – Feladat

<http://ceoi.inf.elte.hu/probarch/09/photo.pdf>

Beadni itt lehet: <https://cses.fi/179/list/>

Adott N pont egy koordináta-rendszerben és egy maximális terület (A).

Cél: találjuk meg a minimális számú téglalapot, amelyek lefedik az összes pontot úgy, hogy:

- minden téglalap egyik oldala az x-tengelyen van,
- minden téglalap területe legfeljebb A ,
- a téglalapok átfedhetnek.

Korlátok:

$$1 \leq N \leq 100, 1 \leq A \leq 200\,000$$

$$0 \leq x_i \leq 3\,000\,000, 1 \leq y_i \leq A$$

photo.in	photo.out	Here is one possible picture that explains the example:
6 4 2 1 4 1 5 1 5 4 7 1 6 4	3	